Bloch-gömb Szimulátor – Dokumentáció

Készítette: Varga Marcell Simon és Borvendég Benedek

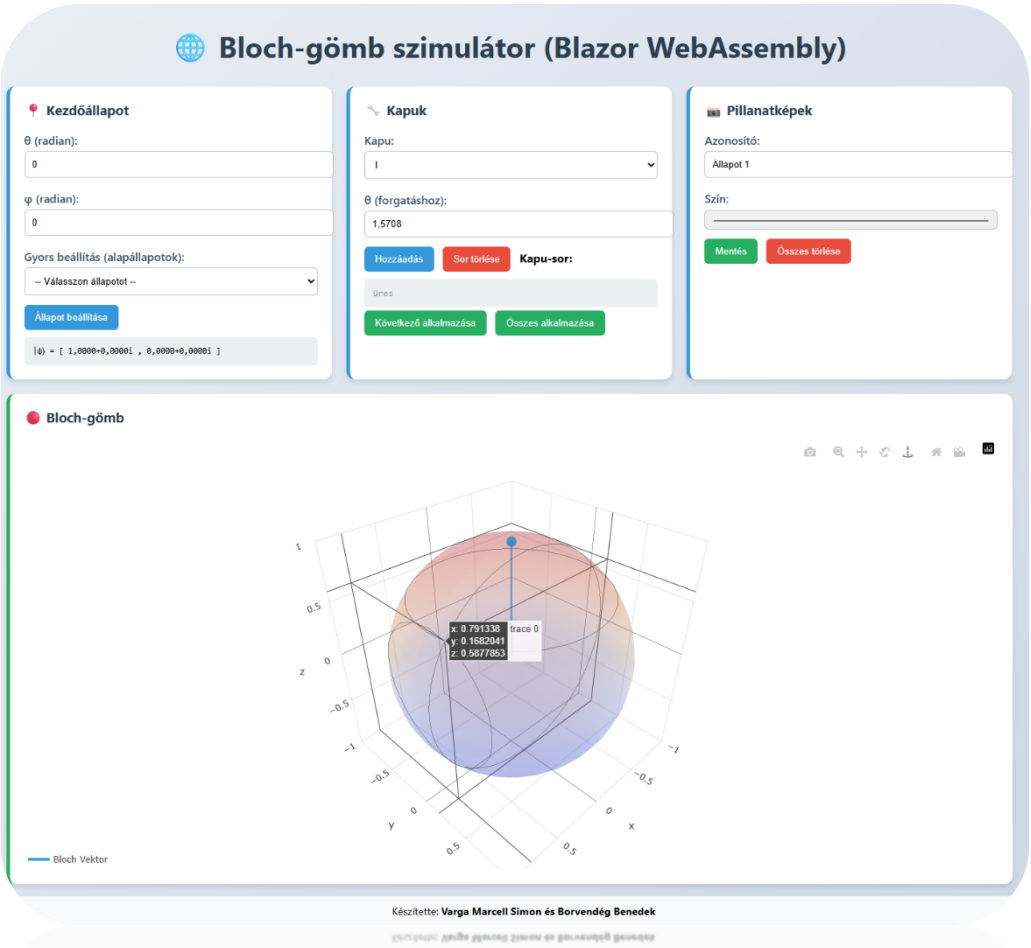
1. Bevezetés

A **Bloch-gömb szimulátor** egy platformfüggetlen, böngészőben futó webalkalmazás, amely interaktív módon teszi lehetővé az egyqubites kvantumállapotok szimulációját és vizualizációját. A projekt célja, hogy a felhasználók vizuálisan, a Bloch-gömb geometriai reprezentációján keresztül érthessék meg a qubit állapotok viselkedését és a kvantumkapuk hatását. Az alkalmazás teljes mértékben kliensoldalon fut (WebAssembly), így nem igényel szerveroldali környezetet.

2. Funkcionális Leírás

Az alkalmazás az alábbi funkciókat kínálja a felhasználóknak, lefedve a kurzus követelményeit:

- **Alapállapotok gyors beállítása:** Hat előre definiált nevezetes állapot közvetlen kiválasztása: $|0\rangle, |1\rangle, |+\rangle, |-\rangle, |i\rangle, |-i\rangle$.
- **Tetszőleges állapot definiálása:** A kvantumállapot precíz beállítása a θ (Theta) és ϕ (Phi) szögek megadásával.
- **Széleskörű kvantumkapu-készlet:**
  - **Pauli-kapuk:** I, X, Y, Z
  - **Speciális kapuk:** H (Hadamard), S (Fázis), T (Pi/8)
  - **Forgatási kapuk:** $Rx(\theta), Ry(\theta), Rz(\theta)$ tetszőleges szöggel.
- **Kapu-sorozatok kezelése:** A felhasználók komplex áramköröket építhetnek fel kapuk egymás után fűzésével. A sorozat bármikor módosítható vagy törölhető.
- **Rugalmas végrehajtás:** Lehetőség van a kapuk hatásának **lépésenkénti** (oktatási célra) vagy **azonnali, teljes** végrehajtására.
- **Pillanatképek (Snapshots) és összehasonlítás:** A felhasználó elmentheti az aktuális állapotokat egyedi címkével és színnel. Ezek az állapotok jelölőpontokként megmaradnak a gömbön, lehetővé téve a kezdő-, köztes- és végállapotok vizuális összehasonlítását.
- **Interaktív 3D Vizualizáció:** A Bloch-gömb szabadon forgatható, nagyítható és mozgatható az egér segítségével.
- **Valós idejű visszajelzés:** Az alkalmazás folyamatosan megjeleníti az aktuális állapotvektor matematikai alakját és a Bloch-koordinátákat.



3. Technológiai Háttér

A projekt modern webes technológiákra épül, biztosítva a hordozhatóságot és a teljesítményt.

3.1 Felhasznált Technológiák és Könyvtárak

Komponens	Technológia / Könyvtár	Verzió	Funkció
Keretrendszer	Blazor WebAssembly	.NET 8.0	C# kód közvetlen futtatása a böngészőben szerver nélkül.
Nyelv	C#	12.0	Üzleti logika, komplex számok kezelése (System.Numerics).
Vizualizáció	Plotly.js	v2.26.0	Nagy teljesítményű 3D grafikus megjelenítés (WebGL).
Stílus	Bootstrap	v5.3.2	Reszponzív elrendezés és modern UI komponensek.

3.2 Architektúra

Az alkalmazás a **Single Page Application (SPA)** modellt követi.

- **Index.razor:** A felhasználói felületet és a vezérlési logikát tartalmazza.
- **QuantumService.cs:** Elkülönített szolgáltatásréteg, amely felelős a kvantumállapot (komplex vektor) tárolásáért, a mátrixműveletekért (kapuk alkalmazása) és a Bloch-vektor számításáért.
- **Interop Réteg:** A C# kód JavaScript hívásokon keresztül kommunikál a Plotly.js könyvtárral a grafikon frissítéséhez.

4. Telepítés és Futtatás

Megjegyzés: A feladat a https://vasutas32.github.io/Bloch-sphere-simulator/ címen elérhető a feladat leírásai szerint, hogy futtatható legyen egy csak böngészővel rendelkező gépről is.

A szoftver egy szabványos .NET 8.0 Blazor WebAssembly projekt, amely Visual Studio környezetben lett fejlesztve.

Futtatás lépései:

1. **Megnyitás:** Csomagolja ki a beadott állományt, majd nyissa meg a Bloch-sphere.sln megoldásfájlt **Visual Studio 2022**-ben.
2. **Indítás:** Győződjön meg róla, hogy a BlochSimulator projekt van beállítva indítóprojektként (jobb klikk a projekten a Solution Explorerben -> *Set as Startup Project*).
3. **Futtatás:** Nyomja meg az **F5** (Start Debugging) vagy a **Ctrl+F5** (Start Without Debugging) billentyűkombinációt.
    - A Visual Studio automatikusan elindítja a beépített IIS Express webszervert, és megnyitja az alapértelmezett böngészőt az alkalmazással.

Megjegyzés: Mivel az alkalmazás kliensoldali technológiát (WebAssembly) használ, a működéshez mindenképpen webszerveren (IIS Express, Kestrel) keresztül kell kiszolgálni az oldalakat; a fájlrendszerből közvetlenül megnyitva (file://) a böngészők biztonsági korlátozásai miatt nem fog működni.

5. Matematikai és Elméleti Háttér

A projekt a kvantummechanika posztulátumaira és a lineáris algebra formalizmusára épül. Jelen fejezet részletezi a szimuláció során alkalmazott matematikai modelleket, az állapotábrázolást és az unitér transzformációkat.

5.1 A Qubit Állapottér

A klasszikus bit bináris értékeivel (0 vagy 1) ellentétben egy kvantumbit (qubit) állapota egy kétdimenziós komplex Hilbert-térben ($\mathcal{H} \cong \mathbb{C}^2$) leírható egységvektor. Az állapotvektor általános alakja a Dirac-jelölés (ket-vektor) használatával:

|= |0+ |1

Ahol $\alpha, \beta \in \mathbb{C}$ a komplex valószínűségi amplitúdók, amelyekre teljesül a normalizálási feltétel:

||^2 + ||^2 = 1

Itt $|0\rangle$ és $|1\rangle$ a számítási bázis (computational basis) ortonormált vektorai:
$|0=\begin{pmatrix}1\\0\end{pmatrix}$, $|1=\begin{pmatrix}0\\1\end{pmatrix}$

5.2 A Bloch-gömb Reprezentáció

Mivel az állapotvektor globális fázisa ($e^{i\gamma}|\psi\rangle$) fizikai mérésekkel nem különböztethető meg, és a normalizálás megköt egy szabadsági fokot, a qubit tiszta állapota leképezhető egy háromdimenziós valós egységgömb, az úgynevezett Bloch-gömb felületére.

A szimulátor a következő transzformációs egyenletekkel számítja ki a Descartes-koordinátákat (x, y, z) a vizualizációhoz:

$$\begin{aligned} x &= \sin(\theta)\cos(\phi) = 2\mathrm{Re}(\alpha\beta^*) \\ y &= \sin(\theta)\sin(\phi) = 2\mathrm{Im}(\alpha\beta^*) \\ z &= \cos(\theta) = |\alpha|^2 - |\beta|^2 \end{aligned}$$

5.3 Kvantumkapuk és Unitér Mátrixok

A zárt kvantumrendszerek időfejlődése unitér operátorokkal írható le. A szimulátorban alkalmazott logikai kapuk 2×2-es unitér mátrixok (U), amelyekre igaz, hogy $U^\dagger U = I$, ahol $U^\dagger$ a mátrix adjungáltja (transzponált konjugáltja).

5.3.1 Pauli-mátrixok

A Pauli-mátrixok a Bloch-gömb tengelyei körüli 180 fokos (π radián) forgatásoknak felelnek meg:

Pauli-X (NOT kapu): Megcseréli a bázisállapotok amplitúdóit. X = $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$, X$\begin{pmatrix}\alpha\\ \beta\end{pmatrix}$ = $\begin{pmatrix}\beta\\ \alpha\end{pmatrix}$

Pauli-Y: Kombinált bit- és fáziscsere. $ Y = $\begin{pmatrix}0 & -i\\ i & 0\end{pmatrix}$

Pauli-Z (Fázis-flip): A $|1\rangle$ állapot fázisát π-vel elforgatja. $ Z = $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, Z$\begin{pmatrix}\alpha\\ \beta\end{pmatrix}$ = $\begin{pmatrix}\alpha\\ -\beta\end{pmatrix}$

5.3.2 Clifford-csoport elemei és egyéb kapuk

Hadamard-kapu (H): A szuperpozíció létrehozásának alapvető eszköze. Transzformálja a Z-bázist az X-bázisba ($|0\rangle \rightarrow |+\rangle$). H = $\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}$

Fázis-kapu (S): A Bloch-vektort 90 fokkal ($\pi/2$) forgatja a Z-tengely körül. S = $\begin{pmatrix}1 & 0\\ 0 & i\end{pmatrix}$

T-kapu: A Bloch-vektort 45 fokkal ($\pi/4$) forgatja a Z-tengely körül. T = $\begin{pmatrix}1 & 0\\ 0 & e^{i\pi/4}\end{pmatrix}$

5.3.3 Általános Forgatási Operátorok

Az alkalmazás lehetővé teszi a Bloch-gömb főtengelyei körüli tetszőleges γ szöggel történő forgatást. Az ezekhez tartozó operátorok az exponenciális leképezéssel származtathatók a Pauli-mátrixokból ($R_{\hat{n}}(\gamma) = \exp\left(-i\frac{\gamma}{2}\hat{n}\cdot\vec{\sigma}\right)$):

X-tengely körüli forgatás: R_x() =$\begin{pmatrix}\cos\frac{\gamma}{2} & -i\sin\frac{\gamma}{2}\\ -i\sin\frac{\gamma}{2} & \cos\frac{\gamma}{2}\end{pmatrix}$

Y-tengely körüli forgatás: R_y() = $\begin{pmatrix}\cos\frac{\gamma}{2} & -\sin\frac{\gamma}{2}\\ \sin\frac{\gamma}{2} & \cos\frac{\gamma}{2}\end{pmatrix}$

Z-tengely körüli forgatás: R_z() = $\begin{pmatrix}e^{-i\gamma/2} & 0\\ 0 & e^{i\gamma/2}\end{pmatrix}$

6. Tesztelés és Validáció

Az alkalmazás helyességét az alábbi tesztekkel igazoltuk:

1. **Nevezetes állapotok:** Ellenőriztük, hogy a gyors beállítások (pl. $|+\rangle$) pontosan a megfelelő tengelyekre (X tengely) kerülnek-e.
2. **Kapu operációk:** Teszteltük a kapuk hatását ismert bemeneteken (pl. $X|0\rangle\rightarrow|1\rangle$, $H|0\rangle\rightarrow|+\rangle$).
3. **Unitaritás:** A szimuláció minden lépésben megőrzi az állapotvektor normáját (a vektor hossza mindig 1 marad).

7. Hivatkozások

1. Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*.
2. Plotly.js Documentation: https://plotly.com/javascript/
3. Microsoft Blazor Documentation: https://learn.microsoft.com/en-us/aspnet/core/blazor/

8. Mesterséges Intelligencia Alkalmazása

A projekt fejlesztése során támogató jelleggel mesterséges intelligencia alapú eszközöket alkalmaztunk. Az AI felhasználása kifejezetten az alábbi területekre korlátozódott:

- **Dokumentáció:** A jelen műszaki leírás szövegezésének, strukturálásának és formázásának segítése.
- **Kódkommentálás:** A forráskódban található magyarázó megjegyzések (kommentek) generálása a kódolvasás és az átláthatóság javítása érdekében.